15: Threat modeling Kubernetes

Security is more important than ever before, and Kubernetes is no exception. Fortunately, there's a lot of things you can do to secure Kubernetes, and you'll see some of them in the next chapter. However, before doing that, it's a good idea to model some of the common threats.

Threat model

Threat modeling is the process of identifying vulnerabilities so you can put measures in place to prevent and mitigate them. This chapter introduces the popular **STRIDE** model and shows how it can be applied to Kubernetes.

STRIDE defines six categories of potential threat:

- Spoofing
- Tampering
- Repudiation
- Information disclosure
- Denial of service
- Elevation of privilege

While the model is good, it's important to keep in mind that it's just a model, and model guarantees to cover all possible threats. However, they are good at providing a structured way to look at things.

For the rest of this chapter, we'll look at each of the six threat categories in turn. For each one, we'll give a quick description, and then look at some of the ways it applies to Kubernetes and how we can prevent and mitigate.

The chapter doesn't try to cover everything. It's just giving you ideas and getting you started.

Spoofing

Spoofing is pretending to be somebody else with the aim of gaining extra privileges on a system.

Let's look at some of the ways Kubernetes prevents different types of spoofing.

Securing communications with the API server

Kubernetes is comprised of lots of small components that work together. These include control plane services such as the API server, controller manager, scheduler, cluster store, and others. It also includes Node components such as the kubelet and container runtime. Each of these has its own set of privileges that allow it to interact with, and even modify the cluster. Even though Kubernetes implements a least-privilege model, spoofing the identity of any of these can cause problems.

If you read the RBAC and API security chapter, you'll know that Kubernetes requires all components to authenticate via cryptographically signed certificates (mTLS). This is good, and Kubernetes makes it easy by auto-rotating certificates and the likes. However, it's vital you consider the following:

1. A typical Kubernetes installation will auto-generate a self-signed certificate authority (CA). This is the CA that will issue certificates to all cluster components. And while it's better than nothing, on its own it's probably not enough for production environments.
2. Mutual TLS (mTLS) is only as secure as the CA issuing the certificates. Compromising the CA can render the entire mTLS layer ineffective. So, keep the CA secure!

A good practice is to ensure that certificates issued by the internal Kubernetes CA are only used and trusted *within* the Kubernetes cluster. This requires careful approval of certificate signing requests, and you need to make sure the Kubernetes CA doesn't get added as a trusted CA for any systems outside of Kubernetes.

As mentioned in previous chapters, all internal and external requests to the API server are subject to authentication and authorization checks. As a result, the API server needs a way to authenticate (trust) internal and external sources. A good way to do this is having two trusted key pairs:

- one for authenticating internal systems
- the other for authenticating external systems

In this model, you'd use the cluster's self-signed CA to issue keys to internal systems. You'd also configure Kubernetes to trust one or more trusted 3rd-party CAs to issue keys to external systems.

Securing Pod communications

As well as spoofing access to the *cluster*, there's also the threat of spoofing an application for app-to-app communications. This is when one Pod spoofs another. Fortunately, you can leverage *Secrets* to mount certificates into Pods that are used to authenticate Pod identity.

While on the topic of Pods, every Pod has an associated `ServiceAccount` that is used to provide an identity for the Pod within the cluster. This is achieved by automatically mounting a service account token into every Pod as a *Secret*. Two points to note:

1. The service account token allows access to the API server
2. Most Pods probably don't need to access the API server

With these two points in mind, it's often recommended to set `automountServiceAccountToken` to false for Pods that you know don't need to communicate with the API server. The following Pod manifest shows how to do this.

```
apiVersion: v1
kind: Pod
metadata:
  name: service-account-example-pod
spec:
  serviceAccountName: some-service-account
  automountServiceAccountToken: false
  <Snip>
```

If the Pod does need to talk to the API server, the following non-default configurations are worth exploring.

- expirationSeconds
- audience

These let you force a time when the token will expire, as well as restrict the entities it works with. The following example, inspired from official Kubernetes docs, sets an expiry period of one hour and restricts it to the `vault` audience in a projected volume.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    volumeMounts:
    - mountPath: /var/run/secrets/tokens
      name: vault-token
  serviceAccountName: my-pod
  volumes:
  - name: vault-token
    projected:
      sources:
      - serviceAccountToken:
          path: vault-token
          expirationSeconds: 3600
          audience: vault
```

Tampering

Tampering is the act of changing something, in a malicious way, so you can cause one of the following.

- Denial of service. Tampering with the resource to make it unusable.
- Elevation of privilege. Tampering with a resource to gain additional privileges.

Tampering can be hard to avoid, so a common counter-measure is to make it obvious when something has been tampered with. A common example, outside of information security, is packaging medication. Most over-the-counter drugs are packaged with tamper-proof seals. These make it easy to see if the product has been tampered with.

Let's have a quick look at some of the cluster components that can be tampered with.

Tampering with Kubernetes components

All of the following Kubernetes components, if tampered with, can cause harm:

- etcd
- Configuration files for the API server, controller-manager, scheduler, etcd, and kubelet
- Container runtime binaries
- Container images
- Kubernetes binaries

Generally speaking, tampering happens either *in transit* or *at rest*. In transit refers to data while it is being transmitted over the network, whereas at rest refers to data stored in memory or on disk.

TLS is a great tool for protecting against *in transit* tampering, as it provides built-in integrity guarantees – you'll be warned if the data has been tampered with.

The following recommendations can also help prevent tampering with data when it is *at rest* in Kubernetes.

- Restrict access to the servers that are running Kubernetes components – especially control plane components.
- Restrict access to repositories that store Kubernetes configuration files.
- Only perform remote bootstrapping over SSH (remember to safely guard your SSH keys).
- Always perform SHA-2 checksums on downloads.
- Restrict access to your image registry and associated repositories.

This isn't an exhaustive list. But if you implement it, you'll greatly reduce the chances of having your data tampered with while at rest.

As well as the items listed, it's good production hygiene to configure auditing and alerting for important binaries and config files. If configured and monitored correctly, these can help detect potential tampering attacks.

The following example uses a common Linux audit daemon to audit access to the `docker` binary. It also audits attempts to the change the binary's file attributes.

```
$ auditctl -w /usr/bin/docker -p wxa -k audit-docker
```

We'll refer to this example later in the chapter.

Tampering with applications running on Kubernetes

As well as infrastructure components, application components are also potential tampering targets.

A good way to prevent a live Pod from being tampered with, is setting its filesystems to read-only. This guarantees filesystem immutability and can be accomplished through a Pod Security Policy or the securityContext section of a Pod manifest file.

> **Note:** PodSecurityPolicy objects allow you to force security settings on all Pods in a cluster, or targeted sub-sets of Pods. They're a great of enforcing standards without developers and operations staff having to remember to do it for every individual Pod.

You can make a container's root filesystem read-only by setting the `readOnlyRootFilesystem` property to `true`. As previously mentioned, this can be set via a PodSecurityPolicy object, or in Pod manifest files. The same can be done for other filesystems that are mounted into containers via the `allowedHostPaths` property.

The following YAML shows how to use both settings in a Pod manifest. The `allowedHostPaths` section makes sure anything mounted beneath `/test` will be read-only.

```
apiVersion: v1
kind: Pod
metadata:
  name: readonly-test
spec:
  securityContext:
    readOnlyRootFilesystem: true
    allowedHostPaths:
      - pathPrefix: "/test"
        readOnly: true
<Snip>
```

The same can be implemented in a PodSecurityPolicy object as follows.

```
apiVersion: policy/v1beta1  # Will change in a future version
kind: PodSecurityPolicy
metadata:
  name: tampering-example
spec:
  readOnlyRootFilesystem: true
  allowedHostPaths:
  - pathPrefix: "/test"
    readOnly: true
```

Repudiation

At a very high level, *repudiation* is creating doubt about something. *Non-repudiation* is providing proof about something. In the context of information security, non-repudiation is **proving** certain actions were carried out by certain individuals.

Digging a little deeper, non-repudiation includes the ability to prove:

- What happened
- When it happened
- Who made it happen

- Where it happened
- Why it happened
- How it happened

Answering the last two usually requires the correlation of several events over a period of time.

Fortunately, auditing of Kubernetes API server events can usually help answer these questions. The following is an example of an API server audit event (you may need to manually enable auditing on your API server).

```
{
  "kind":"Event",
  "apiVersion":"audit.k8s.io/v1",
  "metadata":{ "creationTimestamp":"2020-03-03T10:10:00Z" },
  "level":"Metadata",
  "timestamp":"2020-03-03T10:10:00Z",
  "auditID":"7e0cbccf-8d8a-4f5f-aefb-60b8af2d2ad5",
  "stage":"RequestReceived",
  "requestURI":"/api/v1/namespaces/default/persistentvolumeclaims",
  "verb":"list",
  "user": {
    "username":"fname.lname@example.com",
    "groups":[ "system:authenticated" ]
  },
  "sourceIPs":[ "123.45.67.123" ],
  "objectRef": {
    "resource":"persistentvolumeclaims",
    "namespace":"default",
    "apiVersion":"v1"
  },
  "requestReceivedTimestamp":"2010-03-03T10:10:00.123456Z",
  "stageTimestamp":"2020-03-03T10:10:00.123456Z"
}
```

Although the API server is central to most things in Kubernetes, it's not the only component that requires auditing for non-repudiation. At a minimum, you should collect audit logs from container runtimes, kubelets, and the applications running on your cluster. This is without even mentioning network firewalls and the likes.

Once you start auditing multiple components, you quickly need a centralised location to store and correlate events. A common way to do this is deploying an agent to all nodes via a DaemonSet. The agent collects logs (runtime, kubelet, application…) and ships them to a secure central location.

If you do this, it's vital the centralised log store is secure. If the security of the central log store is compromised, you can no longer trust the logs, and their contents can be repudiated.

To provide non-repudiation relative to tampering with binaries and configuration files, it might be useful to use an audit daemon that watches for write actions on certain files and directories on your Kubernetes Masters and Nodes. For example, earlier in the chapter you saw an example that enabled auditing of changes to the `docker` binary. With this enabled, starting a new container with the `docker run` command will generate an event like this:

```
type=SYSCALL msg=audit(1234567890.123:12345): arch=abc123 syscall=59 success=yes exit=0 a0=12345678abc\
 a1=0 a2=abc12345678 a3=a items=1 ppid=1234 pid=12345 auid=0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 \
s\
gid=0 fsgid=0 tty=pts0 ses=1 comm="docker" exe="/usr/bin/docker" subj=system_u:object_r:container_runt\
ime_exec_t:s0 key="audit-docker"
type=CWD msg=audit(1234567890.123:12345):  cwd="/home/firstname"
type=PATH msg=audit(1234567890.123:12345): item=0 name="/usr/bin/docker" inode=123456 dev=fd:00 mode=0\
100600 ouid=0 ogid=0 rdev=00:00 obj=system_u:object_r:container_runtime_exec_t:s0
```

Audit logs like this, when combined and correlated with Kubernetes' audit features, create a comprehensive and trustworthy picture that cannot be repudiated.

Information Disclosure

Information disclosure is when sensitive data is leaked. There are lots of ways it can happen, including hacked data stores and APIs that unintentionally expose sensitive data.

Protecting cluster data

In the Kubernetes world, the entire configuration of the cluster is stored in the cluster store (usually `etcd`). This includes network and storage configuration, as well as passwords and other sensitive data in Secrets. For obvious reasons, this makes the cluster store a prime target for information disclosure attacks.

As a minimum, you should limit *and* audit access to the nodes hosting the cluster store. As will be seen in the next paragraph, gaining access to a cluster node can allow the logged-on user to bypass some of the security layers.

Kubernetes 1.7 introduced encryption of Secrets but doesn't enable it by default. Even when this becomes default, the data encryption key (DEK) is stored on the same node as the Secret! This means gaining access to a node lets you to bypass encryption. This is especially worrying on nodes that host the cluster store (etcd nodes).

Fortunately, Kubernetes 1.11 enabled a beta feature that lets you store *key encryption keys (KEK)* outside of your Kubernetes cluster. These types of keys are used to encrypt and decrypt data encryption keys and should be safely guarded. You should seriously consider Hardware Security Modules (HSM) or cloud-based Key Management Stores (KMS) for storing your key encryption keys.

Keep an eye on upcoming versions of Kubernetes for further improvements to encryption of Secrets.

Protecting data in Pods

As previously mentioned, Kubernetes has an API resource called a Secret that is the preferred way to store and share sensitive data such as passwords. For example, a front-end container accessing an encrypted back-end database can have the key to decrypt the database mounted as a Secret. This is a far better solution than storing the decryption key in a plain-text file or environment variable.

It is also common to store data and configuration information outside of Pods and containers in Persistent Volumes and ConfigMaps. If the data on these is encrypted, keys for decrypting them should also be stored in Secrets.

With all of this, it's vital that you consider the caveats outlined in the previous section relative to Secrets and how their encryption keys are stored. You don't want to do the hard work of locking the house but leaving the keys in the door.

Denial of Service

Denial of Service (DoS) is all about making something unavailable. There are many types of DoS attack, but a well-known variation is overloading a system to the point it can no longer service requests. In the Kubernetes world, a potential attack might be to overload the API server so that cluster operations grind to a halt (even essential system services have to communicate via the API server).

Let's take a look at some potential Kubernetes systems that might be targets of DoS attacks, and some ways to protect and mitigate.

Protecting cluster resources against DoS attacks

It's a time-honored best practice to replicate essential control plane services on multiple nodes for high availability (HA). Kubernetes is no different, and you should run multiple Masters in an HA configuration for your production environments. Doing this prevents a single Master from becoming a single point of failure. In relation to certain types of DoS attacks, an attacker may need to attack more than one Master to have a meaningful impact.

You should also consider replicating control plane nodes across availability zones. This may prevent a DoS attack on the *network* of a particular availability zone from taking down your entire control plane.

The same principle applies to worker nodes. Having multiple worker nodes not only allows the scheduler to spread your applications over multiple nodes and availability zones, it may also render DoS attacks on any single node or zone ineffective (or less effective).

You should also configure appropriate limits for the following:

- Memory
- CPU
- Storage
- Kubernetes objects

Placing limits on things can help prevent important system resources from being starved, therefore preventing potential DoS.

Limiting Kubernetes objects includes things like; limiting the number of ReplicaSets, Pods, Services, Secrets, and ConfigMaps in a particular Namespace.

Here's an example manifest that limits the number of Pod objects in the `skippy` namespace to 100.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: pod-quota
spec:
  hard:
    pods: "100"
```

One more feature – *podPidsLimit* – restricts the number of processes a Pod can create.

Assume a scenario where a Pod is the target of a fork bomb attack. This is a specialised attack where a rogue process creates as many new processes as possible in an attempt to consume all resources on a system and grind

it to a halt. Placing a limit on the number of processes a Pod can create will prevent the Pod from exhausting the resources of the node and confine the impact of the attack to the Pod. Once the *podPidsLimit* is exhausted, a Pod will typically be restarted.

This also ensures a single Pod doesn't exhaust the PID range for all the other Pods on the node, including the Kubelet. One thing to note though… setting the correct value requires a good estimate of how many Pods will run simultaneously on each Node. Without a ballpark estimate, you can easily over or under allocate PIDs to each pod.

Protecting the API Server against DoS attacks

The API server exposes a RESTful interface over a TCP socket, making it susceptible to botnet-based DoS attacks.

The following may be helpful in either preventing or mitigating such attacks.

- Highly available masters. Having multiple API server replicas running on multiple nodes across multiple availability zones.
- Monitoring and alerting API server requests based on sane thresholds
- Using things like firewalls to limit API server exposure to the internet

As well as botnet DoS attacks, an attacker may also attempt to spoof a user or other control plane service in an attempt to cause an overload. Fortunately, Kubernetes has robust authentication and authorization controls to prevent spoofing. However, even with a robust RBAC model, it's vital that you safeguard access to accounts with high privileges.

Protecting the cluster store against DoS attacks

Cluster configuration is stored in etcd, making it vital that etcd be available and secure. The following recommendations help accomplish this:

- Configure an HA etcd cluster with either 3 or 5 nodes
- Configure monitoring and alerting of requests to etcd
- Isolate etcd at the network level so that only members of the control plane can interact with it

A default installation of Kubernetes installs etcd on the same servers as the rest of the control plane. This is usually fine for development and testing, however, large production clusters should seriously consider a dedicated etcd cluster. This will provide better performance and greater resilience.

On the performance front, etcd is probably the most common choking point for large Kubernetes clusters. With this in mind, you should perform testing to ensure the infrastructure it runs on is capable of sustaining performance at scale – a poorly performing etcd can be as bad as an etcd cluster under a sustained DoS attack. Operating a dedicated etcd cluster also provides additional resilience by protecting it from other parts of the control plane that might be compromised.

Monitoring and alerting of etcd should be based on sane thresholds, and a good place to start is by monitoring etcd log entries.

Protecting application components against DoS attacks

Most Pods expose their main service on the network, and without additional controls in place, anyone with access to the network can perform a DoS attack on the Pod. Fortunately, Kubernetes provides Pod resource request limits to prevent such attacks from exhausting Pod and Node resources. As well as these, the following will be helpful:

- Define Kubernetes Network Policies to restrict Pod-to-Pod and Pod-to-external communications
- Utilize mutual TLS and API token-based authentication for application-level authentication (reject any unauthenticated requests)

For defence in depth, you should also implement application-layer authorization policies that implement least privilege.

Figure 15.1 shows how all of these can be combined to make it hard for an attacker to successfully DoS an application.

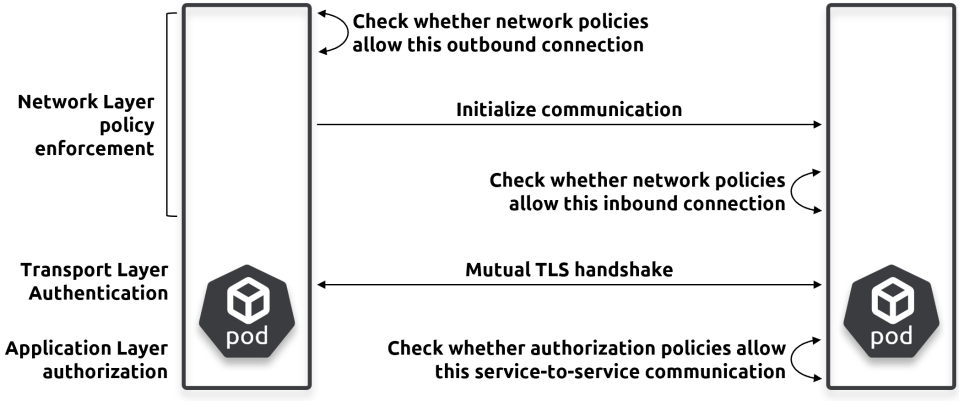

Figure 15.1

Elevation of privilege

Privilege escalation is gaining higher access than what is granted, usually in order to cause damage or gain unauthorized access.

Let's look at a few ways to prevent this in a Kubernetes environment.

Protecting the API server

Kubernetes offers several authorization modes that help safeguard access to the API server. These include:

- Role-based Access Control (RBAC)
- Webhook

- Node

You should run multiple authorizers at the same time. For example, a common best practice is to always have *RBAC* and *node* enabled.

RBAC mode lets you restrict API operations to sub-sets of users. These *users* can be regular user accounts as well as system services. The idea is that all requests to the API server must be authenticated **and** authorized. Authentication ensures that requests are coming from a validated user, whereas authorization ensures the validated user is allowed to perform the requested operation. For example, can *Lily create Pods*? In this example, *Lily* is the user, *create* is the operation, and *Pods* is the resource. Authentication makes sure that it really is Lily making the request, and authorization determines if she's allowed to create Pods.

Webhook mode lets you offload authorization to an external REST-based policy engine. However, it requires additional effort to build and maintain the external engine. It also makes the external engine a potential single-point-of-failure for every request to the API server. For example, if the external webhook system becomes unavailable, you may not be able to make any requests to the API server. With this in mind, you should be rigorous in vetting and implementing any webhook authorization service.

Node authorization is all about authorizing API requests made by kubelets (Nodes). The types of requests made to the API server by Nodes is obviously different to those generally made by regular users, and the node authorizer is designed to help with this.

See the chapter on RBAC and API security for more detail.

Protecting Pods

The next few sections will look at a few of the technologies that help reduce the risk of elevation of privilege attacks against Pods and containers. We'll look at the following:

- Preventing processes from running as `root`
- Dropping capabilities
- Filtering syscalls
- Preventing privilege escalation

As you proceed through the following sections, it's important to remember that a Pod is just an execution environment for one or more containers – application code runs in containers, which in turn, run inside of Pods. Some of the terminology used will refer to Pods and containers interchangeably, but usually we will mean container.

Do not run processes as root

The `root` user is the most powerful user on a Linux system and is always User ID 0 (UID 0). Therefore, running application processes as `root` is almost always a bad idea as it grants the application process full access to the container. This is made even worse by the fact that the `root` user of container often has unrestricted `root` access on the host system as well. If that doesn't make you afraid, nothing will!

Fortunately, Kubernetes lets you force container processes to run as unprivileged non-root users.

The following Pod manifest configures all containers that are part of this Pod to run processes as UID 1000. If the Pod has multiple containers, all processes in all containers will run as UID 1000

```
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  securityContext:  # Applies to all containers in this Pod
    runAsUser: 1000 # Non-root user
  containers:
  - name: demo
    image: example.io/simple:1.0
```

`runAsUser` is one of many settings that can be configured as part of what we refer to as a `PodSecurityContext` (.spec.securityContext).

It's possible for two or more Pods to be configured with the same `runAsUser` UID. When this happens, the containers from both Pods will run with the same security context and potentially have access to the same resources. This *might* be fine if they are replicas of the same Pod or container. However, there's a high chance this will cause problems if they're different containers. For example, two different containers with R/W access to the same host directory or volume can cause data corruption (both writing to the same dataset without co-ordinating write operations). Shared security contexts also increase the possibility of a compromised container tampering with a dataset it shouldn't have access to.

With this in mind, it is possible to use the `securityContext.runAsUser` property at the container level instead of at the Pod level:

```
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  securityContext:  # Applies to all containers in this Pod
    runAsUser: 1000 # Non-root user
  containers:
  - name: demo
    image: example.io/simple:1.0
    securityContext:
      runAsUser: 2000 # Overrides the Pod setting
```

This example sets the UID to 1000 at the Pod level, but overrides it at the container level so that processes in one particular container run as UID 2000. Unless otherwise specified, all other containers in the Pod will use UID 1000.

A couple of other things that might help get around the issue of multiple Pods and containers using the same UID include:

- Enabling *user namespaces*
- Maintaining a map of UID usage

User namespaces is a Linux kernel technology that allows a process to run as `root` within a container, but run as a different user outside of the container. For example, a process can run as UID 0 (the root user) in the container,

but get mapped to UID 1000 on the host. This can be a good solution for processes that *need* to run as root inside the container, but you should check whether it has full support from your version of Kubernetes and your container runtime.

Maintaining a map of UID usage is a clunky way to prevent multiple different Pods and containers using overlapping UIDs. It's a bit of a hack and requires strict adherence to a gated release process for releasing Pods into production.

> **Note:** A strict gated release process is a good thing for production environments. The *hacky* part of the previous section is the UID map itself, as well as the fact that you're introducing an external dependency and complicating releases and troubleshooting.

Drop capabilities

While *user namespaces* allow container processes to run as root inside the container but not on the host machine, it remains a fact that most processes don't really need to run as full root inside the container. However, it is equally true that many processes do require more privileges than a typical non-root user. What is needed, is a way to grant the exact set of privileges a process requires in order to run. Enter *capabilities*.

Time for a quick bit of background…

We've already said the `root` user is the most powerful user on a Linux system. However, its power is a combination of lots of small privileges that we call *capabilities*. For example, the `SYS_TIME` capability allows a user to set the system clock, whereas the `NET_ADMIN` capability allows a user to perform network-related operations such as modifying the local routing table and configuring local interfaces. The root user holds every *capability* and is therefore extremely powerful.

Having a modular set of *capabilities* like this allows you to be extremely granular when granting permissions. Instead of an all or nothing (root or non-root) approach, you can grant a process the exact set of capabilities it requires to run.

There are currently over 30 capabilities, and choosing the right ones can be daunting. With this in mind, an out-of-the-box Docker runtime drops over half of them by default. This is a *sensible-default* that is designed to allow most processes to run, without *leaving the keys in the front door*. While sensible defaults like these are better than nothing, they're often not good enough for a lot of production environments.

A common way to find the absolute minimum set of capabilities an application requires, is to run it in a test environment with all capabilities dropped. This will cause the application to fail and log messages about the missing permissions. You map those permissions to *capabilities,* add them to the application's Pod spec, and run the application again. You rinse and repeat this process until the application runs properly with the minimum set of capabilities.

As good as this is, there are a few things to consider.

Firstly, you **must** perform extensive testing of your application. The last thing you want is a production edge case that you hadn't accounted for in your test environment. Such occurrences can crash your application in production!

Secondly, every fix and update to your application requires the exact same extensive testing against the capability set.

With these considerations in mind, it is vital that you have testing procedures and production release processes that can handle all of this.

By default, Kubernetes implements the default set of *capabilities* implemented by your chosen container runtime (E.g. containerd or Docker). However, you can override this in a Pod Security Policy, or as part of a container's `securityContext` field.

The following Pod manifest shows how to add the `NET_ADMIN` and `CHOWN` capabilities to a container.

```
apiVersion: v1
kind: Pod
metadata:
  name: capability-test
spec:
  containers:
  - name: demo
    image: example.io/simple:1.0
    securityContext:
      capabilities:
        add: ["NET_ADMIN", "CHOWN"]
```

Filter syscalls

seccomp, short for secure computing, is similar in concept to *capabilities* but works by filtering syscalls rather than capabilities.

The way a Linux process asks the kernel to perform an operation is by issuing a *syscall*. seccomp lets you control which syscalls a particular container can make to the host kernel. As with capabilities, a least privilege model is preferred, where the only syscalls a container is allowed to make are the ones it needs to in order to run.

seccomp went GA in Kubernetes 1.19 and can be used in different ways based on the following seccomp profiles.

1. **Non-blocking**: Allows a Pod to run, and records every syscall it makes to an audit log you can use to create a custom profile. The idea is to run your application Pod in a dev/test environment and make it do everything it's designed to do. When you're done, you'll have a log file listing every syscall the Pod needs in order to run. You then use this to create a custom profile that only allows the syscalls the app needs (least privilege).
2. **Blocking**: Blocks all syscalls. It's extremely secure, but prevents a Pod from doing anything useful.
3. **Runtime Default**: Forces a Pod to use the seccomp profile defined by its container runtime. This is a common place to start if you haven't created a custom profile yet. Profiles that ship with container runtimes aren't the most secure in the world, but they're not wide open either. They're usually designed to be balance of *usable* and *secure*, and they're thoroughly tested.
4. **Custom**: A profile that only allows the syscalls your application needs in order to run. Everything else is blocked. It's common to extensively test your application in dev/test with a non-blocking profile that records all syscalls to an audit log. You then use this log to identify the syscalls your app makes and build the customized profile. The danger with this approach is that your app has some edge-cases that you miss with your testing. If this happens, your application can fail in production when it hits an edge-cases and uses a syscall not captured during testing.

Obviously, custom profiles operate the *least privilege* model and are the preferred approach from a security perspective.

Prevent privilege escalation by containers

The only way to create a new process in Linux is for one process to clone itself and then load new instructions on to the new process. We're obviously over-simplifying, but the original process is called the *parent* process, and the copy is called the *child*.

By default, Linux allows a *child* process to claim more privileges than its *parent*. This is usually a bad idea. In fact, you'll often want a child process to have the same, or less privileges than its parent. This is especially true for containers, as their security configurations are defined against their initial configuration, and not against potentially escalated privileges.

Fortunately, it's possible to prevent privilege escalation through a PodSecurityPolicy or the securityContext property of an individual container.

The following Pod manifest shows how to prevent privilege escalation for an individual container.

```
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
  - name: demo
    image: example.io/simple:1.0
    securityContext:
      allowPrivilegeEscalation: false
```

Pod Security Policies

As you've seen throughout the chapter, you can enable security settings on a per-Pod basis by setting security context attributes in individual Pod YAML files. However, this approach doesn't scale, it requires developers and operators to remember to do this for every Pod, and is prone to errors. *Pod Security Policies* offer a better way.

Pod Security Policies allow you to define security settings at the cluster level. You can then apply them to targeted sets of Pods as part of the deployment process. This approach scales better, requires less effort from developers and admins, and is less prone to error. It also lends itself to situations where you have a team dedicated to securing apps in production.

Pod Security Policies are implemented as an *admission controller,* and in order to use them, a Pod's ServiceAccount must be authorized to use it. Once this is done, policies are applied to new requests to create Pods as they pass through the API admission chain.

Pod Security Policy example

Let's finish the chapter with a quick look at an example of a Pod Security Policy that covers many of the points discussed in this chapter, as well as some other known secure defaults.

It's based on an example from the official Kubernetes docs[1]:

[1] https://kubernetes.io/docs/concepts/policy/pod-security-policy/#example-policies

```
apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: restricted
  annotations:
    seccomp.security.alpha.kubernetes.io/allowedProfileNames: 'docker/default'
    apparmor.security.beta.kubernetes.io/allowedProfileNames: 'runtime/default'
    seccomp.security.alpha.kubernetes.io/defaultProfileName:  'docker/default'
    apparmor.security.beta.kubernetes.io/defaultProfileName:  'runtime/default'
spec:
  privileged: false
  allowPrivilegeEscalation: false  # Prevent privilege escalation
  requiredDropCapabilities:
    - ALL # Drops all root capabilities (non-privileged user)
  # Allow core volume types.
  volumes:
    - 'configMap'
    - 'emptyDir'
    - 'projected'
    - 'secret'
    - 'downwardAPI'
    # Assume that PVs set up by the cluster admin are safe to use.
    - 'persistentVolumeClaim'
  hostNetwork: false # Prevent access to the host network namespace
  hostIPC: false # Prevent access to the host IPC namespce
  hostPID: false # Prevent access to the host PID namespace
  runAsUser:
    rule: 'MustRunAsNonRoot' # Prevent from running as root
  runAsGroup:
    rule: 'MustRunAs' # controls which primary Group ID containers are run with
    ranges:
      - min: 1
        max: 65535
  seLinux:
    rule: 'RunAsAny' # Any SELinux options can be used
  supplementalGroups:
    rule: 'MustRunAs' # Allow all except root (UID 0)
    ranges:
      - min: 1
        max: 65535
  fsGroup:
    rule: 'MustRunAs' # Sets range for groups that own Pod volumes
    ranges:
      - min: 1
        max: 65535
  readOnlyRootFilesystem: true # Force root filesystem to be R/O
  forbiddenSysctls:
  - '*' #Forbids any sysctls from being accessible from a pod
```

There's no denying that configuring effective security policies is both important and challenging. A common practice is to start with a restrictive policy like the one just shown, then tweak it to fit your requirements. A lot

of experimenting will be required.

It may also be a good idea to configure several Pod Security Policies that vary in how restrictive they are, then allow development teams to work with cluster administrators to choose the one that best fits the application.

Towards more secure Kubernetes

In 2019, the CNCF (Cloud Native Computing Foundation) commissioned a third-party security audit of Kubernetes. There were several findings, including threat modeling, manual code reviews, dynamic penetration testing, and a cryptography review. All findings were given a difficulty and severity level, and all high severity findings were fixed prior to the release of the report. You can find the report here: http://shorturl.at/stwxH

The 2020 "Cloud Native Security Whitepaper" is also of interest, as is the Kubernetes perspective provided by this book's co-author Pushkar Joglekar on the official Kubernetes blog: http://shorturl.at/evQ48

Reports like thses can be a great way to learn more about Kubernetes and how the internals work. Studying reports like these is a great way to level-up after reading this chapter.

Chapter summary

In this chapter, you saw how the STRIDE model can be used to threat model Kubernetes. You stepped through the six categories of threat, and looked at some ways to prevent and mitigate them.

You saw that one threat can often lead to another, and that there are multiple ways to mitigate a single threat. As always, defence in depth is a key tactic.

The chapter finished by discussing how Pod Security Policies provide a flexible and scalable way to implement Pod security defaults.

In the next chapter, you'll see some best practices and lessons learned from running Kubernetes in production.